National University of Computer and Emerging Sciences

Islamabad Campus

Web Programming (CS4032)

Course Instructor(s):
Dr.Tajwar Mehmood
Section(s): (A,B & C)

Final Exam

Total Time (Hrs): 1
Total Marks: 60
Total Questions: 1

Date: May 23, 2026

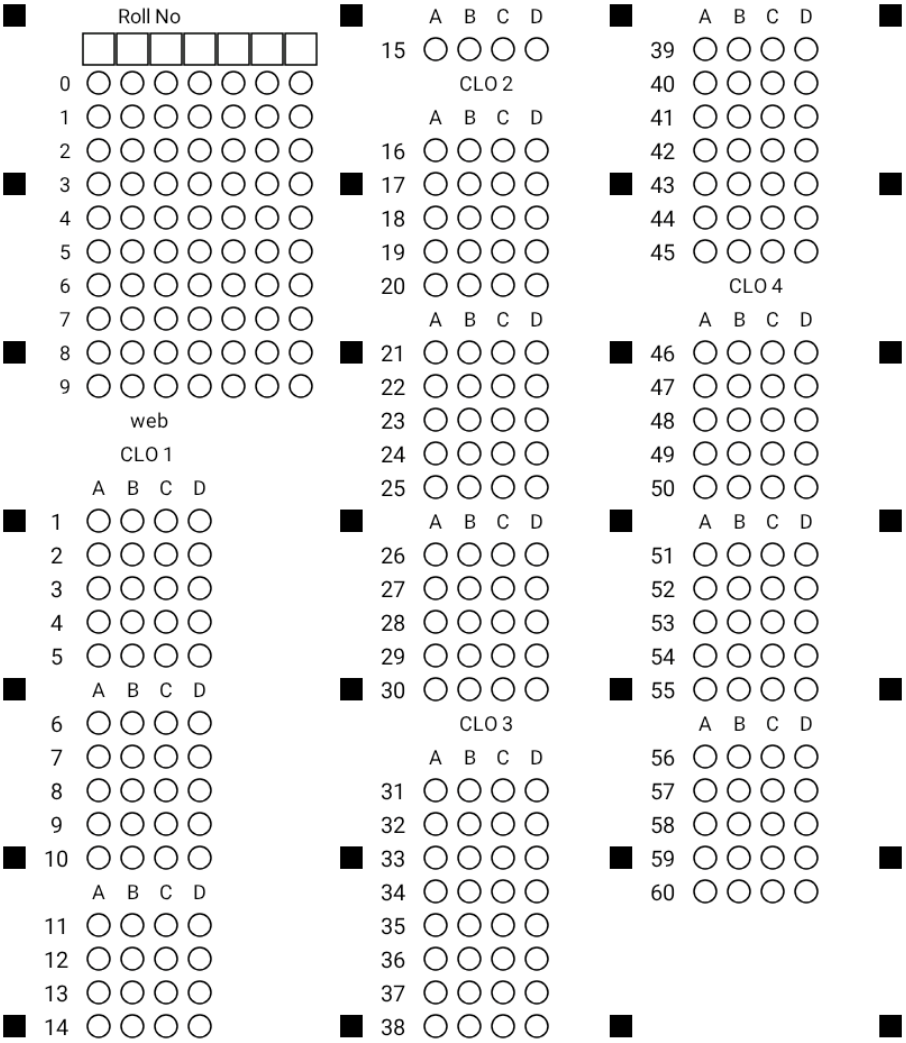
Roll No **Course Section** **Student Signature**

Important Information: **The use or possession of mobile phones and other electronic devices during the exam is strictly prohibited, whether the device is powered on or off. Do not bring mobile phones, smartwatches, tablets, or other electronic gadgets into the examination hall. If you are found with any prohibited devices, your exam may be cancelled, and further disciplinary action may be taken.**

Attempt all the questions.

Part 1

Note: A **one-page cheat sheet is allowed** during the assessment.



National University of Computer and Emerging Sciences
Islamabad Campus

CLO 1: Develop and design web applications using modern web development frameworks
CLO 2: Learn how to use different web frameworks to create a complete industry-oriented project using MEAN and MERN Stack
CLO 3: Able to design, develop, and deploy full stack web applications
CLO 4: Work in a team to complete enterprise projects and professional ethics and responsibilities.

Cheat sheet ☺

Concept	Hint	Concept	Hint
localStorage	survives browser close	sessionStorage	removed on tab close
null	missing key	axios.get+params	query string
axios.post(data)	body	axios.delete({data})	can send body
setState async	old value logs first	rerender	UI updates automatically
props: parent→child	data flow	useEffect([])	run once
useEffect([x])	run on x change	return()=>{}	cleanup/unmount
key=index ✕	bad if reorder/remove	`_id`	Mongo auto field
`$set`	update only field	middleware	before request
git push	upload commits	git pull	fetch + merge
`.env → .gitignore`	hide secrets	Hydration	server ≠ client HTML
next/image	optimize images	router.push()	navigate programmatically

Question # 1: MCQs
CLO 1

Question 1: What is printed to the console?

```
localStorage.setItem('user', 'Ali');
console.log(localStorage.getItem());
```

A) null
B) Ali
C) undefined
D) user

Question 2: What does this code do compared to a simple document.cookie = 'theme=dark'?

```
document.cookie = 'theme=dark; expires=Thu, 01 Jan 2099 00:00:00 UTC; path=/';
```

A) It deletes the existing cookie named theme
B) It sets a persistent cookie with an expiry date and path scope
C) It creates a session cookie that expires when the browser closes
D) It throws a SyntaxError because of the extra attributes

Question 3: What does the first .then(res => res.json()) call return?

```
fetch('/api/data')
  .then(res => res.json())
  .then(data => console.log(data));
```

A) The print JSON object immediately
B) Another Promise that resolves to the parsed JSON
C) A string of raw JSON
D) The HTTP status code

Question 4: How will 'role=admin' be sent in this request?

```
axios.get('/users', { params: { role: 'admin' } })
```

A) In the request body as JSON
B) As a custom HTTP header

C) As a query string: /users?role=admin
D) It will be ignored — axios.get does not support params

Question 5: What happens when the browser tab is closed and reopened?

```
sessionStorage.setItem('token', '123');  // Tab is closed
console.log(sessionStorage.getItem('token'));
```

A) The token persists and logs '123'
B) The token is cleared; getItem returns null
C) The token moves to localStorage automatically
D) An error is thrown on getItem

Question 6: What does the third argument '2' control in JSON.stringify?

```
const obj = { name: 'Ali', age: 20 };// Hint: Focus on how the output looks when printed in the console. The third argument affects the spacing and formatting of the JSON text, making it easier for humans to read.
console.log(JSON.stringify(obj, null, 2));
```

A) The maximum depth of serialization
B) The indentation spaces for pretty-printing
C) The number of properties to include
D) The JSON version format

Question 7: What does typeof data.age return?

```
const data = JSON.parse('{"age":"20"}');
console.log(typeof data.age);
```

A) number
B) string
C) object
D) undefined

Question 8: What is logged?

```
const p = Promise.resolve(5);
p.then(v => console.log(v));
```

A) Promise { 5 }
B) 5
C) undefined — .then must be awaited
D) Error: Promise already resolved

Question 9: Where is { name: 'Ali' } sent in this request?

```
axios.post('/login', { name: 'Ali' });
```

A) As a query string parameter
B) In the request body as JSON
C) As a custom header
D) It is not sent — axios.post ignores the second argument

Question 10: What is logged? Hint: After removeItem() is called, the key no longer exists in sessionStorage. Think about what getItem() returns for a missing key.

```
sessionStorage.setItem('token', 'abc');
sessionStorage.removeItem('token');
console.log(sessionStorage.getItem('token'));
```

A) abc
B) undefined
C) null
D) Error

Question 11: What is the output? Hint: fetch is a built-in browser API used to make HTTP requests.

```
console.log(typeof fetch);
```

A) object
B) function
C) promise
D) string

Question 12: What does localStorage.length return after clear()?

```
localStorage.setItem('a', '1');
localStorage.setItem('b', '2');
localStorage.clear();
console.log(localStorage.length);
```

A) 2
B) 1
C) 0
D) undefined

Question 13: What is true about this axios call?

```
axios.delete('/users/1', { data: { reason: 'inactive' } });
```

A) axios.delete cannot send a body; the data option is ignored
B) It sends a DELETE request with a JSON body containing reason
C) It sends a GET request because delete is not a valid HTTP method in axios
D) It requires a then() to execute

Question 14: What does document.cookie log?

```
document.cookie = 'user=Ali';
document.cookie = 'role=admin';
console.log(document.cookie);
```

A) role=admin — the second assignment overwrites the first
B) user=Ali
C) user=Ali; role=admin
D) An array of cookie objects

Question 15: What is logged?

```
fetch('/api')
  .then(res => { throw new Error('oops'); })
  .catch(err => console.log(err.message));
```

A) Nothing — .catch only handles network errors
B) oops
C) undefined
D) Error object is not accessible in .catch

CLO 2:

Question 16: What happens each time the button is clicked?

```
function Counter() {
  const [count, setCount] = React.useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

A) count increments but the UI does not update until the page refreshes
B) React re-renders Counter, displaying the new count value
C) count is incremented globally across all components
D) setCount triggers a full page reload

Question 17: What is logged immediately by console.log(count) after these two setCount calls (assuming React batching)?

```
const [count, setCount] = useState(0);
setCount(count + 1);
setCount(count + 1);
console.log(count);
```

A) 2
B) 1
C) 0 — state updates are asynchronous
D) Error

Question 18: Why is using the array index as a key generally discouraged?

```
const users = ['Ali', 'Sara'];
return users.map((u, i) => <li key={i}>{u}</li>);
```

A) Indexes are not valid JSX key values
B) If items are reordered or removed, keys no longer uniquely identify elements, causing rendering bugs
C) map() does not support two arguments in JSX
D) It forces a full DOM re-render on every update

Question 19: Which field does MongoDB automatically add to this document?

```
db.users.insertOne({ name: 'Ali', age: 20 });
```

A) id
B) _id with an ObjectId
C) uuid
D) No field is added automatically

Question 20: What does this MongoDB query return?

```
db.users.find({ age: { $gt: 18 } });
```

A) Users whose age equals 18
B) Users whose age is greater than 18
C) Users whose age field exists
D) An error — $gt is not a valid operator

Question 21: What is logged?

```
const user = { name: 'Ali', address: { city: 'Lahore' } };
console.log(user.address.city)
```

A) undefined
B) Ali
C) Lahore

D) Error

Question 22: How does the component receive 'Sara'?

```
function Greeting({ name }) {
  return <h1>Hello, {name}</h1>;
}
<Greeting name="Sara" />
```

A) Via component state
B) Via a cookie
C) Via props passed from the parent
D) Via a React context

Question 23: Why must JSX be wrapped in a single root element (like)?

```
return (
  <div>
    <h1>Title</h1>
    <p>Text</p>
  </div>
);
```

A) JSX is converted to React.createElement calls, which return a single element
B) HTML requires a single root node inside JavaScript
C) Multiple roots cause CSS to break
D) This is a React 18 requirement only

Question 24: What does $set do in this query?

```
// MongoDB
db.users.updateOne(
  { name: 'Ali' },
  { $set: { age: 25 } }
);
```

A) Replaces the entire document with { age: 25 }
B) Updates only the age field, leaving other fields intact
C) Creates a new document if none match
D) Deletes the age field

Question 25: When does this useEffect run?

```
useEffect(() => {
  fetchData();
}, [userId]);
```

A) Only once when the component mounts
B) On every render
C) Whenever userId changes, and once on mount
D) Only when the component unmounts

Question 26: What is the relationship between App and Child?

```
function App() {
  return <Child />;
}
function Child() {
```

```
  return <p>Hello</p>;
}
```

A) Child is a sibling of App
B) App is a parent component that renders Child
C) Child passes props to App
D) They share state by default

Question 27: What is the purpose of the return function inside useEffect?

```
useEffect(() => {
  const id = setInterval(() => console.log('tick'), 1000);
  return () => clearInterval(id);
}, []);
```

A) It re-runs the effect after every render
B) It is a cleanup function that runs when the component unmounts
C) It prevents the effect from running on mount
D) It is ignored by React

Question 28: What is the difference between 'export default' and 'export'?

```
export default App;
```

A) No difference — both are identical
B) 'export default' exports one value per module; named 'export' can export multiple
C) 'export default' only works in React files
D) 'export' is required for components; 'export default' is for utilities

Question 29: How many props is MyButton receiving?

```
<MyButton color="blue" onClick={handleClick} />
```

A) 1
B) 2
C) 3
D) 0 — attributes are not props in JSX

Question 30: In a MERN stack, which letter does this Express code represent?

```
// MERN Stack
const express = require('express');
const app = express();
app.get('/users', (req, res) => res.json(users));
```

A) M — MongoDB
B) E — Express
C) R — React
D) N — Node.js

CLO3:

Question 31: What URL does this file create in Next.js App Router?

```
// app/about/page.js
export default function About() {
  return <h1>About</h1>;
}
```

A) /about/page
B) /app/about

C) /about
D) /page

Question 32: When does getServerSideProps execute?

```
export async function getServerSideProps(context) {
  const data = await fetchData(context.params.id);
  return { props: { data } };
}
```

A) Once at build time
B) On every incoming request on the server
C) In the browser after hydration
D) Only when the user logs in

Question 33: What does 'use client' enable in Next.js App Router?

```
'use client';
import { useState } from 'react';
```

A) Server-side data fetching
B) Client-side interactivity including hooks and browser APIs
C) Static site generation
D) API route handling

Question 34: This is a Next.js Server Component. Where does the fetch() call run?

```
// app/dashboard/page.js
async function Dashboard() {
  const data = await fetch('https://api.example.com/stats').then(r => r.json());
  return <div>{data.total}</div>;
}
```

A) In the browser after the page loads
B) On the server — the component fetches data server-side before sending HTML to the client
C) In a Web Worker thread
D) Only during the build step

Question 35: What kind of routing does [id] enable in Next.js?

```
// pages/blog/[id].js
export default function Post({ id }) {}
```

A) Static routing — id must be defined in next.config.js
B) Dynamic routing — id is captured from the URL
C) Catch-all routing — id matches any number of segments
D) API routing — id is a query parameter name

Question 36: What advantage does next/link have over a regular <a> tag?

```
import Link from 'next/link';
<Link href="/about">About</Link>
```

A) It loads faster by skipping CSS parsing
B) It enables client-side navigation without a full page reload
C) It automatically adds HTTPS to the href
D) It prevents the browser from caching the page

Question 37: What does this file do in Next.js App Router?

```
// app/not-found.js
export default function NotFound() {
  return <h2>Page Not Found</h2>;
}
```

A) It is a custom 404 page shown when a route is not matched
B) It is a layout that wraps all pages
C) It disables routing for the /not-found path
D) It handles server errors (500)

Question 38: Why must 'use client' be present for this to work?

```
'use client';
useEffect(() => {
  document.title = 'Hello';
}, []);
```

A) useEffect is a server-only API in Next.js
B) document and useEffect require a browser environment; server components cannot use them
C) document.title is deprecated in Next.js
D) useEffect only runs inside API routes

Question 39: What is the URL of this API route?

```
// pages/api/hello.js
export default function handler(req, res) {
  res.status(200).json({ msg: 'Hello' });
}
```

A) /pages/hello
B) /hello
C) /api/hello
D) /api/handler

Question 40: What advantage does next/image provide over a plain <img> tag?

```
import Image from 'next/image';
<Image src="/photo.jpg" width={500} height={300} alt="Photo" />
```

A) It converts images to SVG automatically
B) It provides automatic lazy loading, resizing, and modern format (WebP) conversion
C) It removes the need for alt text
D) It stores images in MongoDB

Question 41: What problem does this scenario describe?

```
// Hydration
// Server renders: <div>Count: 0</div>
// Client expects: <div>Count: 1</div>
```

A) A CORS error
B) A hydration mismatch, where server and client HTML differ
C) A missing useEffect cleanup
D) An incorrect API route

Question 42: What is the role of layout.js in Next.js App Router?

```
// app/layout.js
export default function RootLayout({ children }) {
```

```
  return <html><body>{children}</body></html>;
}
```

A) It defines a shared UI wrapper that persists across route changes
B) It replaces getServerSideProps
C) It is the entry point for API routes
D) It configures Tailwind CSS

Question 43: What does loading.js do in Next.js App Router?

```
// app/dashboard/loading.js
export default function Loading() {
  return <p>Loading...</p>;
}
```

A) It runs before the server starts
B) It shows a loading UI automatically while the page or layout is fetching data
C) It replaces the layout.js file
D) It configures webpack loading rules

Question 44: What does router.push('/dashboard') do?

```
import { useRouter } from 'next/navigation';
const router = useRouter();
router.push('/dashboard');
```

A) Adds /dashboard as a new API route
B) Navigates to /dashboard programmatically without a page reload
C) Refreshes the current page
D) Pushes data to the server

Question 45: Where does Next.js middleware execute?

```
// Middleware
export function middleware(req) {
  if (!req.cookies.token) return NextResponse.redirect('/login');
}
```

A) In the browser before the component renders
B) On the Edge/server before the request reaches the page or API route
C) After the response is sent to the client
D) Inside getServerSideProps only

CLO4:

Question 46: What does this line do?

```
localStorage.setItem('jwt', token);
```

A) Saves the token in the browser's localStorage
B) Sends the token to the server
C) Deletes the existing token
D) Creates a new JWT

Question 47: What does this line do?

```
const token = localStorage.getItem('jwt');
```

A) Deletes the JWT from localStorage
B) Updates the JWT value
C) Reads the JWT stored in localStorage

D) Creates a new JWT token

Question 48: What does JWT stand for?

A) Java Web Tool
B) JSON Web Token
C) JavaScript Worker Thread
D) JSON Worker Type

Question 49: What is JWT mainly used for in web apps?

A) Styling web pages
B) Storing images
C) User authentication and authorization
D) Connecting to a database

Question 50: What does 'git add .' do?

```
git add .
```

A) Pushes changes to GitHub
B) Deletes all files
C) Creates a new branch
D) Stages all changed files for commit

Question 51: What does this command do?

```
git commit -m "add login page"
```

A) Pushes code to GitHub
B) Saves a snapshot of staged changes with a message
C) Creates a new branch
D) Merges two branches

Question 52: What does this command do?

```
git push origin main
```

A) Downloads changes from the remote repo
B) Deletes the main branch
C) Uploads local commits to the remote repository
D) Creates a new branch called origin

Question 53: What does 'git pull' do?

```
git pull
```

A) Uploads your code to GitHub
B) Fetches and merges the latest changes from the remote repo
C) Deletes the latest commit
D) Creates a new local branch

Question 54: Why do we add .env to .gitignore?

```
// .gitignore
.env
node_modules/
```

A) To make .env load faster
B) To keep secret keys and passwords from being pushed to GitHub
C) Because .env files are too large
D) Git does not support .env files

Question 55: What does 'vercel deploy' do?

```
vercel deploy
```

A) Deletes the deployed app
B) Installs project dependencies
C) Publishes your app to the internet via Vercel
D) Runs your app locally

Question 56: What is Vercel mainly used for?

```
// Vercel is used for:
```

A) Writing CSS styles
B) Managing databases
C) Hosting and deploying web applications
D) Creating JWT tokens

Question 57: Why is this a good commit message?

```
// Good commit message:
git commit -m "fix: correct login redirect bug"
```

A) It is very short
B) It clearly describes what was changed and why
C) It uses random words
D) It skips staging

Question 58: What is wrong with this commit message?

```
// Bad commit message:
git commit -m "stuff"
```

A) It is too descriptive
B) Commit messages cannot contain letters
C) It gives no useful information about what was changed
D) It will cause a merge conflict

Question 59: Why should you never share your API keys publicly?

```
// Ethical practice: Never share your API keys publicly
```

A) API keys slow down the server when shared
B) Shared keys can be misused by others, leading to unauthorized access or charges
C) API keys only work on one computer
D) GitHub automatically deletes shared keys

Question 60: What is the main benefit of using Git for version control?

A) It makes your CSS look better
B) It connects your app to a database
C) It tracks code changes and allows you to revert to previous versions
D) It automatically fixes bugs

Q No	KEY
1	A
2	B
3	B
4	C
5	B
6	B
7	B
8	B
9	B
10	C
11	B
12	C
13	B
14	C
15	B
16	B
17	C
18	B
19	B
20	B
21	C
22	C
23	A
24	B
25	C
26	B
27	B
28	B
29	B
30	B

Q No	KEY
31	C
32	B
33	B
34	B
35	B
36	B
37	A
38	B
39	C
40	B
41	B
42	A
43	B
44	B
45	B
46	A
47	C
48	B
49	C
50	D
51	B
52	C
53	B
54	B
55	C
56	C
57	B
58	C
59	B
60	C